

COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.
- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.

- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

Submitted by:

T Deekshith (192472253)

1. Problem Understanding and Formulation

1.1 Problem Statement

CampusConnect is considered a university digital platform through which students and faculty access shared academic resources such as notes, assignments, reports, images, database records and large lecture videos. Because many users may access the platform at the same time, the operating system must coordinate processes, memory and storage resources safely and efficiently.

The objective of this assignment is to develop an integrated operating-system resource-management design that provides deadlock-free concurrent access, efficient memory utilization, reliable file storage and fair, low-overhead disk access.

1.2 Synchronization Challenges

Multiple CampusConnect users may access the same academic file simultaneously. Multiple readers can safely read a file at the same time, whereas a writer must obtain exclusive access. Without synchronization, simultaneous reads and writes can create race conditions, inconsistent data, partial updates and file corruption.

- Shared-file access must be mutually coordinated.
- Readers should be allowed to operate concurrently.
- A writer must have exclusive access.
- The synchronization mechanism should avoid race conditions and starvation as far as practical.

1.3 Deadlock Challenges

CampusConnect processes may require multiple resources, including a database handle, print spooler, network socket and file lock. Deadlock is possible when processes hold some resources while waiting indefinitely for resources held by other processes.

P1 holds Database Handle → waits for File Lock
P2 holds File Lock → waits for Database Handle

Result: P1 and P2 wait indefinitely unless the system prevents or avoids the unsafe allocation.

1.4 Memory-Management Challenges

With approximately 300 active sessions, memory pressure can become significant. If processes receive too few frames, pages may be repeatedly loaded from and evicted to disk. Excessive page faults can cause thrashing and reduce CPU utilization.

1.5 File-System Challenges

CampusConnect has a mixed workload ranging from small text documents and PDFs to large recorded lectures. The allocation strategy must support file growth, sequential and random access, and efficient use of storage without excessive fragmentation.

1.6 Disk-Scheduling Challenges

Hundreds of user operations can generate simultaneous I/O requests. Inefficient scheduling can increase head movement and seek time. FCFS, SCAN and C-SCAN are therefore compared using a representative request queue.

1.7 Expected Outcomes

- Deadlock-free and race-condition-free shared access.
- Efficient physical-memory utilization with reduced page faults.
- Thrashing prevention during high concurrent load.
- Efficient storage for small files and multi-GB videos.
- Reduced disk-head movement and fair access to I/O requests.
- Reliable and responsive access to shared academic resources.

1.8 Assumptions and Justification

Parameter	Assumed Value	Justification
Physical memory	8 GB	Given in the problem statement.
Page size	4 KB	Given in the problem statement and common OS page size.
Representative process	64 MB	Given in the problem statement.
Concurrent sessions	300	Given in the problem statement.
Page-replacement frames	3	Small fixed frame count for clear algorithm comparison.
Disk cylinders	0–199	Given as 200 cylinders.
Initial disk head	53	Representative mid-disk starting position.
Banker processes	P1–P4	Given in the problem statement.
Resource types	4	Database handle, print spooler, network socket, file lock.
PTE size	8 bytes	Reasonable assumption for page-table-size calculation.

2. Application of Course Knowledge

Part I – Process Synchronization & Deadlock (CO2, Unit II)

4. Readers–Writers Model for Shared Files

The shared academic-file problem is modeled as the classical Readers–Writers problem. Readers can access the file concurrently, but a writer requires exclusive access. A mutex protects the read-count variable and a write semaphore protects the file from simultaneous writing.

4.1 Semaphore Pseudocode

Pseudocode 1 – Semaphore-based Readers–Writers synchronization

```
READER:
    wait(mutex)
    read_count = read_count + 1
    if read_count == 1:
        wait(write)
    signal(mutex)

    READ FILE
```

```
wait(mutex)
read_count = read_count - 1
if read_count == 0:
    signal(write)
signal(mutex)
```

WRITER:

```
wait(write)
```

WRITE FILE

```
signal(write)
```

Logic: the first reader locks the writer semaphore, allowing other readers to enter while preventing writers. The last reader releases the writer semaphore. A writer holds the write semaphore throughout its critical section, so readers and other writers cannot access the file simultaneously.

4.2 Python Synchronization Simulation

Python implementation – Readers–Writers synchronization

```
import threading
import time
import random

read_count = 0
mutex = threading.Semaphore(1)
write_lock = threading.Semaphore(1)

shared_file = "Initial CampusConnect academic content"

def reader(reader_id):
    global read_count

    mutex.acquire()
    read_count += 1
    if read_count == 1:
        write_lock.acquire()
    mutex.release()

    print(f'Reader {reader_id} started reading')
    print("Content:", shared_file)
    time.sleep(random.uniform(0.5, 1.5))
    print(f'Reader {reader_id} finished reading')

    mutex.acquire()
    read_count -= 1
    if read_count == 0:
        write_lock.release()
    mutex.release()

def writer(writer_id):
```

```

global shared_file

write_lock.acquire()
print(f'Writer {writer_id} started writing')
shared_file = f'Updated CampusConnect content by Writer {writer_id}'
time.sleep(1)
print(f'Writer {writer_id} finished writing')
write_lock.release()

threads = []

for i in range(3):
    threads.append(threading.Thread(target=reader, args=(i + 1,)))

for i in range(2):
    threads.append(threading.Thread(target=writer, args=(i + 1,)))

for t in threads:
    t.start()

for t in threads:
    t.join()

print("\nFinal Shared File:")
print(shared_file)

```

5. Banker's Algorithm Safety Check

Four resource types are assumed: A = database handle, B = print spooler, C = network socket, and D = file lock. The following allocation and maximum matrices are constructed as a reasonable example for four processes.

5.1 Allocation Matrix

Process	A	B	C	D
P1	1	0	1	0
P2	1	1	0	1
P3	1	0	1	1
P4	0	1	1	0

5.2 Maximum Matrix

Process	A	B	C	D
P1	2	1	1	1
P2	2	1	1	1
P3	2	1	2	1
P4	1	1	1	1

Total resources are [5, 3, 4, 3]. Total allocation is [3, 2, 3, 2], giving Available = Total – Allocation = [2, 1, 1, 1].

5.3 Need Matrix

Need = Maximum – Allocation.

Process	A	B	C	D
P1	1	1	0	1
P2	1	0	1	0
P3	1	1	1	0
P4	1	0	0	1

5.4 Safety Algorithm Trace

Step	Process Selected	Need $\leq$ Work?	Work After Completion
Initial	—	—	[2,1,1,1]
1	P1	Yes	[3,1,2,1]
2	P2	Yes	[4,2,2,2]
3	P3	Yes	[5,2,3,3]
4	P4	Yes	[5,3,4,3]

Safe sequence: P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P4. Since every process can complete and release its allocated resources, the system is in a SAFE state.

Python implementation – Banker's Algorithm

```
import numpy as np

allocation = np.array([
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [1, 0, 1, 1],
    [0, 1, 1, 0]
])

maximum = np.array([
    [2, 1, 1, 1],
    [2, 1, 1, 1],
    [2, 1, 2, 1],
    [1, 1, 1, 1]
])

available = np.array([2, 1, 1, 1])
need = maximum - allocation

finish = [False] * len(allocation)
safe_sequence = []
work = available.copy()

print("Need Matrix:\n", need)
print("\nInitial Available:", work)

while len(safe_sequence) < len(allocation):
    found = False
    for i in range(len(allocation)):
        if not finish[i] and np.all(need[i] <= work):
            print(f"\nP{i+1} can execute.")
            print("Need:", need[i])
```

```

print("Available before:", work)
work += allocation[i]
print("Available after:", work)

finish[i] = True
safe_sequence.append(i + 1)
found = True

if not found:
    break

if len(safe_sequence) == len(allocation):
    print("\nSYSTEM IS SAFE")
    print("Safe Sequence:",
          " -> ".join(f"P{x}" for x in safe_sequence))
else:
    print("\nSYSTEM IS NOT SAFE")

```

6. Deadlock-Handling Strategy

A combination of deadlock avoidance and resource-ordering rules is recommended for CampusConnect. Banker's Algorithm is appropriate for bounded resource pools where maximum demands can be estimated. For dynamic resources such as network connections, timeouts and detection mechanisms can complement avoidance.

Strategy	Advantage	Limitation	CampusConnect Assessment
Prevention	Can eliminate a necessary deadlock condition.	May reduce concurrency or resource utilization.	Useful for strict lock-order rules.
Avoidance	Prevents unsafe allocations before they occur.	Requires maximum-demand information.	Recommended for bounded critical resources.
Detection & Recovery	Flexible resource allocation.	Deadlock may occur before recovery.	Useful as a secondary safety mechanism.
Ignore	Very low overhead.	Unsafe for critical academic services.	Not recommended.

Part II – Memory Management (CO3, Unit III)

7. Paging Calculation

Physical memory = 8 GB and page size = 4 KB.

8 GB = 2 ³³ bytes 4 KB = 2 ¹² bytes Number of frames = 2 ³³ / 2 ¹² = 2 ²¹ = 2,097,152 frames

Therefore, the 8 GB physical memory contains 2,097,152 frames of 4 KB each.

For a 64 MB logical address space: 64 MB = 2²⁶ bytes. Number of pages = 2²⁶ / 2¹² = 2¹⁴ = 16,384 pages.



Figure 1. Logical address structure for the 64 MB process.

A 4 KB page requires a 12-bit offset. The remaining 14 bits identify one of 16,384 pages, giving a 26-bit logical address.

Assuming an 8-byte page-table entry, page-table size = $16,384 \times 8 = 131,072$ bytes = 128 KB.

7.1 Page Table Entry Contents

- Frame number
- Valid/present bit
- Protection bits (read/write/execute)
- Dirty/modified bit
- Reference/access bit
- Other OS-specific control information

7.2 Two-Level Page Table Design

The 14-bit page number can be split into two 7-bit indices. The logical address can therefore be viewed as Level-1 index (7 bits) + Level-2 index (7 bits) + offset (12 bits). This allows second-level tables to be allocated only for portions of the address space that are actually used.

8. Page Replacement: FIFO, LRU and Optimal

Reference string used for the representative user session: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. The comparison uses 3 frames.

8.1 Results

Algorithm	Page Faults	Main Principle	Practicality
FIFO	10	Replace the page that entered first.	Simple; may show Belady's anomaly.
LRU	9	Replace the least recently used page.	Practical approximation possible.
Optimal	7	Replace the page used farthest in the future.	Theoretical benchmark; future references unavailable.



Figure 2. Page-fault comparison for the selected reference string.

Python implementation – Page replacement algorithms

```
reference = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
frames_count = 3

def fifo(reference, frame_count):
    frames, queue, faults = [], [], 0
    for page in reference:
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
                queue.append(page)
            else:
                oldest = queue.pop(0)
                frames[frames.index(oldest)] = page
                queue.append(page)
    return faults

def lru(reference, frame_count):
    frames, recent, faults = [], {}, 0
    for position, page in enumerate(reference):
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
            else:
                victim = min(frames, key=lambda x: recent[x])
                frames[frames.index(victim)] = page
            recent[page] = position
```

```

return faults

def optimal(reference, frame_count):
    frames, faults = [], 0
    for i, page in enumerate(reference):
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
            else:
                future = reference[i+1:]
                def next_use(p):
                    return future.index(p) if p in future else float("inf")
                victim = max(frames, key=next_use)
                frames[frames.index(victim)] = page
    return faults

print("FIFO:", fifo(reference, frames_count))
print("LRU:", lru(reference, frames_count))
print("Optimal:", optimal(reference, frames_count))

```

9. Demand Paging and Working-Set Strategy

Demand paging loads a page only when it is referenced. This avoids loading unused portions of every process into RAM and therefore improves memory utilization.

```

Process starts
↓
Only required pages loaded
↓
Page fault occurs when an absent page is referenced
↓
OS loads the page from storage
↓
Execution continues

```

For 300 concurrent sessions, the working-set model should be used to estimate each process's active pages over a recent reference window. If a process has a high page-fault rate, additional frames can be allocated when available. If its working set shrinks, frames can be reclaimed.

A practical policy is to reserve approximately 1 GB for the OS kernel, file/page cache and background services, leaving approximately 7 GB for user sessions. Equal division would provide roughly 1,835,008 user frames, or about 6,117 frames per session (approximately 23.9 MB). This should be treated as a capacity reference rather than a fixed allocation.

Policy Level	Approx. Frames	Approx. Memory	Use
Minimum	2,048	8 MB	Baseline allocation for an active session.
Normal target	4,096–6,000	16–23.4 MB	Typical interactive session range.
Adaptive	Above normal when needed	Based on available RAM	Increase for high page-fault/working-set demand.

If the combined working-set demand exceeds available memory, low-priority or inactive sessions should be throttled or suspended rather than allowing all processes to thrash.

Part III – File Systems & Disk Scheduling (CO4, Unit IV)

10. File Allocation Methods

CampusConnect has a mixed file-size workload. Contiguous, linked and indexed allocation are compared below.

Property	Contiguous	Linked	Indexed
Sequential access	Excellent	Good	Excellent
Random access	Excellent	Poor	Excellent
External fragmentation	Yes	No	No
File growth	Difficult	Easy	Easy
Pointer overhead	Low	High	Moderate
Mixed file-size workload	Poor–Moderate	Moderate	Excellent

10.1 Recommended Method

Indexed allocation is recommended because it supports dynamic file growth and efficient random access without requiring contiguous free space. It is suitable for PDFs, assignments and large video files. In modern implementations, extent-based indexing can further reduce metadata overhead for large sequential files.

11. Disk Scheduling

Disk cylinders are 0–199. The request queue is 98, 183, 37, 122, 14, 124, 65, 67 and the initial head position is 53. For SCAN and C-SCAN, the initial direction is toward higher-numbered cylinders.

11.1 FCFS Calculation

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

Movement = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2

Total = 640 cylinders

11.2 SCAN Calculation

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14

Total head movement = 331 cylinders

11.3 C-SCAN Calculation

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37

Total head movement = 382 cylinders

Algorithm	Total Head Movement	Observation
FCFS	640 cylinders	Highest movement; simple arrival-order fairness.
SCAN	331 cylinders	Lowest for this workload; good fairness/performance balance.

C-SCAN	382 cylinders	More uniform service direction; extra wrap-around movement.
--------	---------------	---

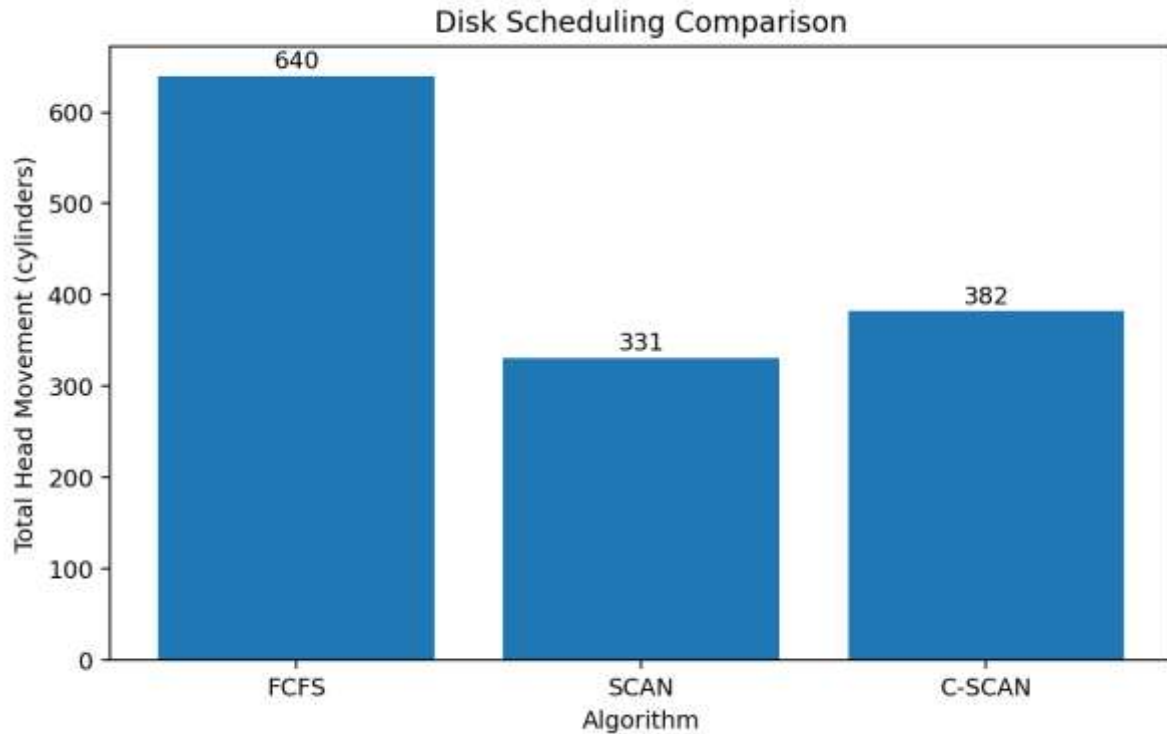


Figure 3. Disk-scheduling comparison for the selected request queue.

Python implementation – Disk scheduling algorithms

```

requests = [98, 183, 37, 122, 14, 124, 65, 67]
head = 53
MAX_CYLINDER = 199

def fcfs(requests, head):
    total, current, path = 0, head, [head]
    for request in requests:
        total += abs(current - request)
        current = request
        path.append(request)
    return total, path

def scan(requests, head):
    lower = sorted([x for x in requests if x < head])
    higher = sorted([x for x in requests if x >= head])
    sequence = higher + [MAX_CYLINDER] + list(reversed(lower))
    total, current, path = 0, head, [head]
    for request in sequence:
        total += abs(current - request)
        current = request
        path.append(request)
    return total, path

def cscan(requests, head):

```

```

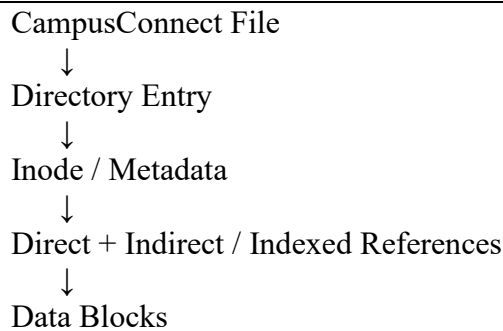
lower = sorted([x for x in requests if x < head])
higher = sorted([x for x in requests if x >= head])
sequence = higher + [MAX_CYLINDER, 0] + lower
total, current, path = 0, head, [head]
for request in sequence:
    total += abs(current - request)
    current = request
    path.append(request)
return total, path

for name, fn in [("FCFS", fcfs), ("SCAN", scan), ("C-SCAN", cscan)]:
    total, path = fn(requests, head)
    print(name)
    print("Path:", path)
    print("Total Movement:", total, "cylinders\n")

```

12. UNIX File-System Concepts

The recommended indexed-allocation approach can be related to UNIX-like inode structures. An inode stores file metadata and references to data blocks. Traditional inode designs use direct and indirect block pointers, which are conceptually related to indexed allocation.



A buffer or page cache keeps frequently accessed blocks in RAM. If a student requests a frequently used PDF, the system can serve cached blocks without immediately accessing storage, reducing I/O latency.

3. Solution / Design / Methodology

3.1 Consolidated Design

The final CampusConnect design combines synchronization, memory management and storage management into one resource-management pipeline.

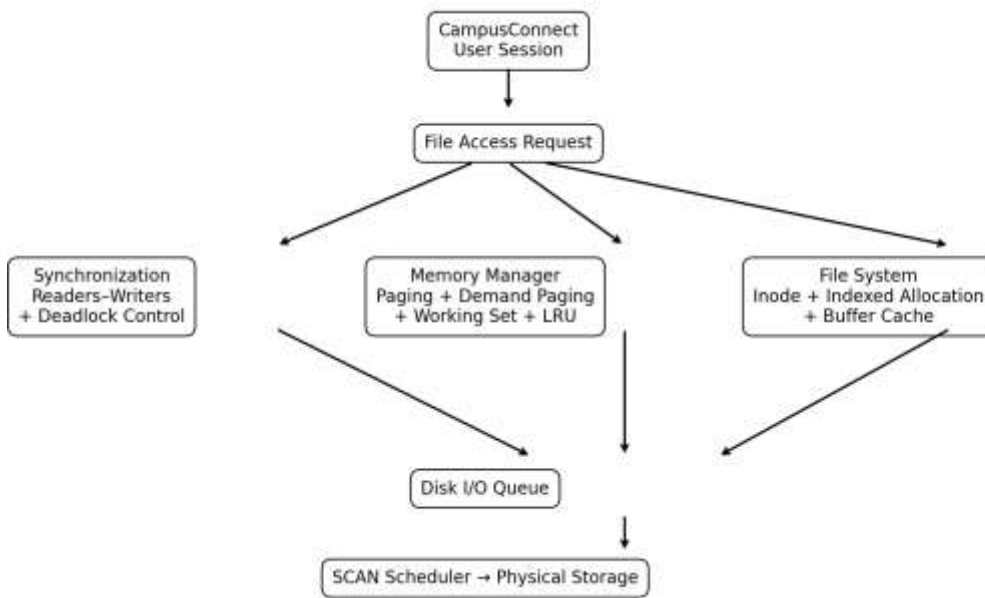


Figure 4. Integrated CampusConnect operating-system resource-management architecture.

3.2 Synchronization Design

- Use Readers–Writers synchronization for shared academic files.
- Protect read_count with a mutex.
- Use a writer semaphore for exclusive modification.
- Apply consistent lock ordering to reduce circular-wait risk.
- Use bounded waits/timeouts for external or dynamically allocated resources.

3.3 Memory-Management Design

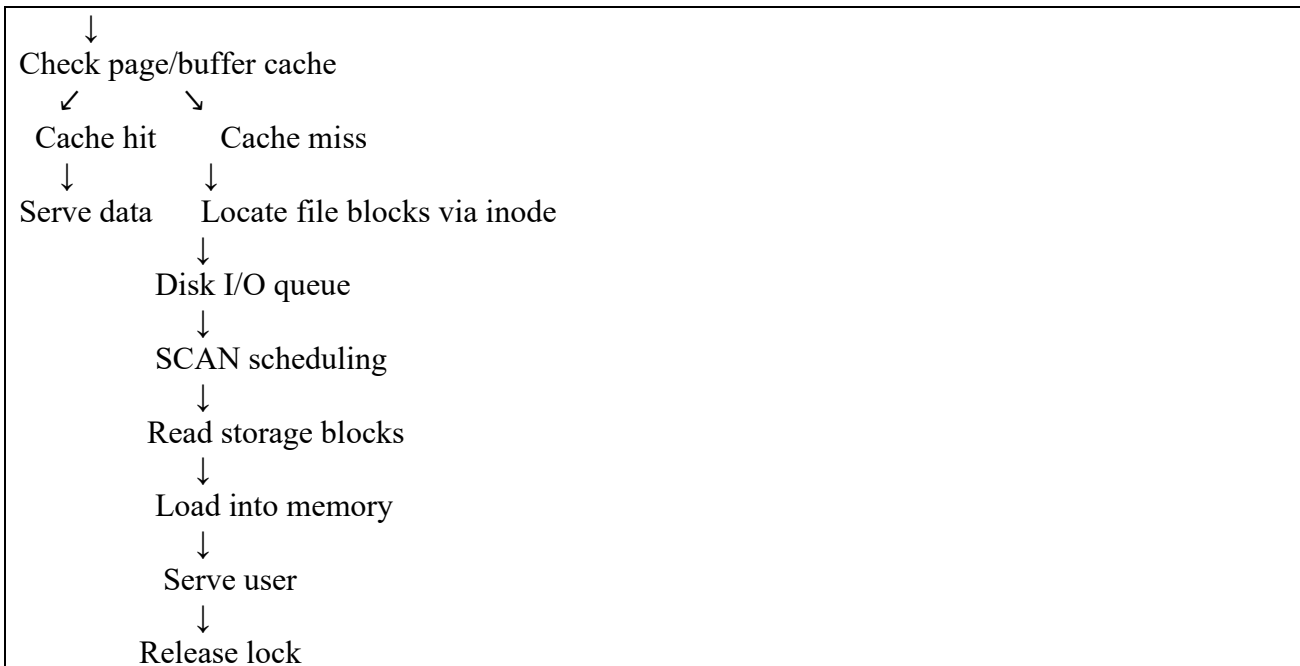
- Use 4 KB paging.
- Use demand paging to load only referenced pages.
- Use LRU or a practical LRU approximation such as Clock/Second-Chance.
- Track working-set size and page-fault frequency.
- Keep a free-frame reserve and avoid allocating more total working-set demand than physical memory.

3.4 File-System and Disk Design

- Use indexed or extent-based allocation for mixed file sizes.
- Use inode-style metadata and block references.
- Use buffer/page cache for frequently accessed academic files.
- Use SCAN for the representative mechanical-disk workload because it produced the lowest measured movement.

3.5 User Session File-Access Flow

User requests file ↓ Authentication / permission check ↓ Acquire reader/writer synchronization lock



4. Use of Modern Tools and Implementation

Google Colab is suitable for the Python simulations because it provides a browser-based Python environment and allows the source code and outputs to be captured directly for the report.

Tool	Purpose
Google Colab	Run and demonstrate Python implementations.
Python	Implement OS algorithms and simulations.
NumPy	Matrix operations in Banker's Algorithm.
Matplotlib	Generate comparison graphs.
threading + Semaphore	Demonstrate Readers–Writers synchronization.
Linux / WSL (optional)	Cross-check semaphore behavior with POSIX threads.

4.1 Google Colab Execution Order

1. Run the Readers–Writers synchronization cell and capture the console output.
2. Run the Banker's Algorithm cell and capture the Need Matrix, safety trace and safe sequence.
3. Run the page-replacement cell and capture FIFO, LRU and Optimal page-fault counts.
4. Run the page-fault graph cell and capture the comparison figure.
5. Run the disk-scheduling cell and capture paths and head-movement totals.
6. Run the disk graph cell and capture the comparison figure.

4.2 Page-Fault Graph Code

```

import matplotlib.pyplot as plt

algorithms = ['FIFO', 'LRU', 'Optimal']
faults = [10, 9, 7]

plt.figure(figsize=(7, 5))
plt.bar(algorithms, faults)

```



```
plt.xlabel('Page Replacement Algorithm')
plt.ylabel('Number of Page Faults')
plt.title('Page Replacement Comparison')
plt.show()
```

4.3 Disk-Scheduling Graph Code

```
import matplotlib.pyplot as plt

algorithms = ['FCFS', 'SCAN', 'C-SCAN']
movement = [640, 331, 382]

plt.figure(figsize=(7, 5))
plt.bar(algorithms, movement)
plt.xlabel('Disk Scheduling Algorithm')
plt.ylabel('Total Head Movement')
plt.title('Disk Scheduling Comparison')
plt.show()
```

5. Results and Validation

5.1 Banker's Algorithm Validation

The safety check produced the safe sequence $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$. All four processes can complete under the assumed maximum-demand matrix, so the state is SAFE.

5.2 Page-Replacement Results

Algorithm	Faults	Rank for This Workload
Optimal	7	1 – Best theoretical result
LRU	9	2 – Recommended practical policy
FIFO	10	3 – Simple baseline



Figure 5. Page-fault results.

5.3 Disk-Scheduling Results

Algorithm	Movement	Reduction vs FCFS
FCFS	640	0%
SCAN	331	48.28%
C-SCAN	382	40.31%

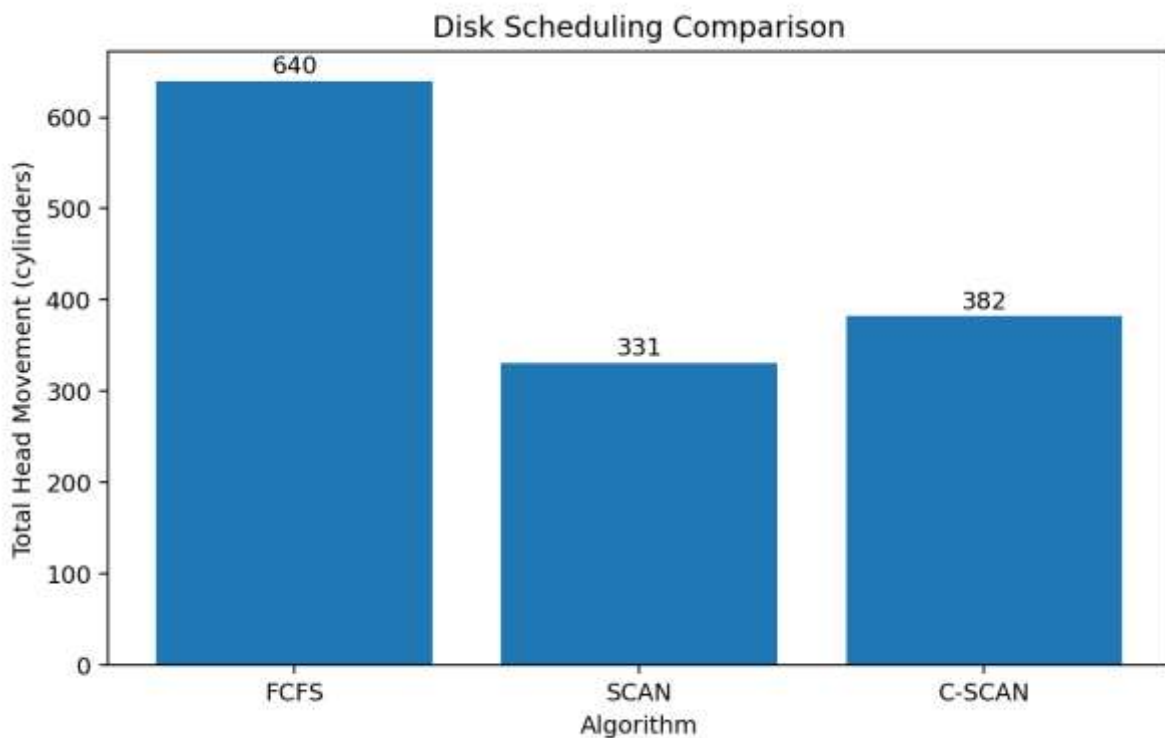


Figure 6. Disk head-movement results.

5.4 Synchronization Validation

The synchronization simulation uses a mutex around `read_count` and a write semaphore around the file's write critical section. Multiple readers can execute concurrently, while a writer waits until no readers are active and then obtains exclusive access.

- No simultaneous writer/writer access.
- No reader/writer overlap while the writer semaphore is held.
- `read_count` updates are protected by mutex.
- The demonstration should be stress-tested with more threads for stronger validation.

6. Analysis and Engineering Decision

6.1 Page-Replacement Decision

Optimal produced 7 faults and is therefore the theoretical lower bound for the chosen reference string. However, it requires knowledge of future references and cannot be implemented exactly in a real OS. LRU produced 9 faults and better reflects locality of reference, making an LRU approximation a practical choice.

6.2 Disk-Scheduling Decision

FCFS is simple and preserves arrival order but produced 640 cylinders of movement. SCAN reduced movement to 331 cylinders, a reduction of approximately 48.28%. C-SCAN produced 382 cylinders and provides more uniform service in one direction. For the selected workload, SCAN offers the best measured balance between fairness and seek efficiency.

6.3 Deadlock Strategy Decision

Avoidance is preferred for bounded resources because it checks whether a resource request leaves the system in a safe state. Prevention through a global lock order should also be applied where possible because it is simple and reduces circular-wait risk. Detection and recovery can be retained as a secondary mechanism for unexpected dynamic resource conditions.

6.4 Overall Engineering Decision

Subsystem	Selected Design	Reason
Synchronization	Readers–Writers + semaphores	Safe concurrent reads and exclusive writes.
Deadlock	Avoidance + lock ordering + timeout support	Balances reliability and concurrency.
Memory	Paging + demand paging + LRU approximation + working set	Reduces faults and prevents thrashing.
File system	Indexed / extent-based allocation + inode concepts	Handles mixed file sizes and random access.
Disk	SCAN	Lowest movement in the selected workload with good fairness.

7. Broader Considerations

7.1 Reliability and Safety

- Use file locks and atomic update operations to protect academic data.
- Use authentication and role-based authorization for student and faculty resources.

- Maintain backups and recovery procedures.
- Use integrity checks and audit logs for important files.
- Protect metadata and cached data from unauthorized modification.

7.2 Sustainability and Economics

Efficient page allocation reduces unnecessary paging and storage I/O. Buffer caching reduces repeated disk reads. Lower disk-head movement reduces mechanical seek work for HDD-based storage. Together these measures can reduce resource usage, improve throughput and potentially reduce hardware and energy costs.

7.3 Accessibility and Society

Fair synchronization and disk scheduling help prevent a small group of users from monopolizing shared resources. Working-set-aware memory management helps maintain responsiveness during peak periods, while admission control can prevent system-wide thrashing.

7.4 Ethics and Professional Responsibility

CampusConnect may process student records, grades, assignments and personal information. The design should follow least privilege, secure authentication, confidentiality, appropriate retention, auditing and responsible handling of academic data. AI-assisted development should be disclosed according to institutional rules, and generated code should be tested rather than accepted without verification.

8. Conclusion

This report presented an integrated operating-system resource-management design for CampusConnect. Readers–Writers synchronization protects shared academic files, while deadlock avoidance and lock-ordering rules reduce the possibility of unsafe resource allocation.

The memory analysis showed that 8 GB physical memory with 4 KB pages provides 2,097,152 frames, while a 64 MB logical address space contains 16,384 pages. The representative page-replacement experiment produced 10 FIFO faults, 9 LRU faults and 7 Optimal faults. LRU or a practical LRU approximation is therefore recommended because it captures locality without requiring future knowledge.

For storage, indexed or extent-based allocation is appropriate for the mixed workload. The disk experiment produced 640 cylinders for FCFS, 331 for SCAN and 382 for C-SCAN. SCAN was selected for the representative mechanical-disk workload because it provided the lowest head movement while retaining reasonable fairness.

The final design therefore satisfies the major stated requirements: safe concurrency, efficient memory utilization, thrashing control, scalable file storage, fair disk access, and improved reliability. A production implementation would require real workload traces, security testing, failure testing and evaluation on modern SSD/NVMe storage.

9. Student Reflection

9.1 What I Learned

This assignment helped me understand that synchronization, memory management and file systems are interconnected rather than isolated OS topics. A single user request can involve a process lock, page allocation, cache lookup, file metadata and disk scheduling. The practical simulations also demonstrated why a theoretically optimal algorithm is not always the best real-world choice.

9.2 Future Improvements

- Use real CampusConnect workload traces instead of manually constructed sequences.
- Test hundreds or thousands of concurrent processes.

- Compare LRU with Clock/Second-Chance and adaptive page-fault-frequency policies.
- Implement POSIX semaphores and pthreads in a Linux/WSL environment.
- Evaluate SSD/NVMe behavior, where traditional mechanical head scheduling is less relevant.
- Use replication, distributed storage and automatic failover for large-scale deployment.
- Perform security, fault-injection and stress testing.

10. References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2015.
- [3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed., Pearson, 2018.
- [4] IEEE, IEEE Standard 1003.1, POSIX Threads and synchronization interfaces.
- [5] Python Software Foundation, Python Documentation: threading and synchronization primitives.
- [6] NumPy Developers, NumPy Documentation.
- [7] Matplotlib Development Team, Matplotlib Documentation.
- [8] Google, Google Colaboratory Documentation.
- [9] OpenAI, ChatGPT, used as an AI-assisted tool for explanation, drafting and code guidance; all calculations and outputs were independently checked through implementation.

Appendix A – Complete Google Colab Code

The following code blocks can be copied into separate Google Colab cells. The same source code is also provided in the relevant sections above for easier documentation and screenshot capture.

A.1 Banker's Algorithm

```
import numpy as np

allocation = np.array([
    [1, 0, 1, 0],
    [1, 1, 0, 1],
    [1, 0, 1, 1],
    [0, 1, 1, 0]
])

maximum = np.array([
    [2, 1, 1, 1],
    [2, 1, 1, 1],
    [2, 1, 2, 1],
    [1, 1, 1, 1]
])

available = np.array([2, 1, 1, 1])
need = maximum - allocation

finish = [False] * len(allocation)
safe_sequence = []
work = available.copy()

print("Need Matrix:\n", need)
print("\nInitial Available:", work)

while len(safe_sequence) < len(allocation):
```

```

found = False
for i in range(len(allocation)):
    if not finish[i] and np.all(need[i] <= work):
        print(f"\nP {i+1} can execute.")
        print("Need:", need[i])
        print("Available before:", work)
        work += allocation[i]
        print("Available after:", work)

        finish[i] = True
        safe_sequence.append(i + 1)
        found = True

if not found:
    break

if len(safe_sequence) == len(allocation):
    print("\nSYSTEM IS SAFE")
    print("Safe Sequence:",
          " -> ".join(f"P {x}" for x in safe_sequence))
else:
    print("\nSYSTEM IS NOT SAFE")

```

A.2 Page Replacement

```

reference = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
frames_count = 3

def fifo(reference, frame_count):
    frames, queue, faults = [], [], 0
    for page in reference:
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
                queue.append(page)
            else:
                oldest = queue.pop(0)
                frames[frames.index(oldest)] = page
                queue.append(page)
    return faults

def lru(reference, frame_count):
    frames, recent, faults = [], {}, 0
    for position, page in enumerate(reference):
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
            else:
                victim = min(frames, key=lambda x: recent[x])

```

```

        frames[frames.index(victim)] = page
    recent[page] = position
    return faults

def optimal(reference, frame_count):
    frames, faults = [], 0
    for i, page in enumerate(reference):
        if page not in frames:
            faults += 1
            if len(frames) < frame_count:
                frames.append(page)
            else:
                future = reference[i+1:]
                def next_use(p):
                    return future.index(p) if p in future else float("inf")
                victim = max(frames, key=next_use)
                frames[frames.index(victim)] = page
    return faults

print("FIFO:", fifo(reference, frames_count))
print("LRU:", lru(reference, frames_count))
print("Optimal:", optimal(reference, frames_count))

```

A.3 Disk Scheduling

```

requests = [98, 183, 37, 122, 14, 124, 65, 67]
head = 53
MAX_CYLINDER = 199

def fcfs(requests, head):
    total, current, path = 0, head, [head]
    for request in requests:
        total += abs(current - request)
        current = request
        path.append(request)
    return total, path

def scan(requests, head):
    lower = sorted([x for x in requests if x < head])
    higher = sorted([x for x in requests if x >= head])
    sequence = higher + [MAX_CYLINDER] + list(reversed(lower))
    total, current, path = 0, head, [head]
    for request in sequence:
        total += abs(current - request)
        current = request
        path.append(request)
    return total, path

def cscan(requests, head):
    lower = sorted([x for x in requests if x < head])
    higher = sorted([x for x in requests if x >= head])

```

```

sequence = higher + [MAX_CYLINDER, 0] + lower
total, current, path = 0, head, [head]
for request in sequence:
    total += abs(current - request)
    current = request
    path.append(request)
return total, path

for name, fn in [("FCFS", fcfs), ("SCAN", scan), ("C-SCAN", cscan)]:
    total, path = fn(requests, head)
    print(name)
    print("Path:", path)
    print("Total Movement:", total, "cylinders\n")

```

A.4 Readers–Writers

```

import threading
import time
import random

read_count = 0
mutex = threading.Semaphore(1)
write_lock = threading.Semaphore(1)

shared_file = "Initial CampusConnect academic content"

def reader(reader_id):
    global read_count

    mutex.acquire()
    read_count += 1
    if read_count == 1:
        write_lock.acquire()
    mutex.release()

    print(f'Reader {reader_id} started reading')
    print("Content:", shared_file)
    time.sleep(random.uniform(0.5, 1.5))
    print(f'Reader {reader_id} finished reading')

    mutex.acquire()
    read_count -= 1
    if read_count == 0:
        write_lock.release()
    mutex.release()

def writer(writer_id):
    global shared_file

    write_lock.acquire()
    print(f'Writer {writer_id} started writing')

```



```

shared_file = f'Updated CampusConnect content by Writer {writer_id}'
time.sleep(1)
print(f'Writer {writer_id} finished writing')
write_lock.release()

threads = []

for i in range(3):
    threads.append(threading.Thread(target=reader, args=(i + 1,)))

for i in range(2):
    threads.append(threading.Thread(target=writer, args=(i + 1,)))

for t in threads:
    t.start()

for t in threads:
    t.join()

print("\nFinal Shared File:")
print(shared_file)

```

A. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20
Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

C. Faculty Evaluation & Continuous

Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____ Areas

requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

Github Repository: <https://github.com/leviii-tech/CSA6501-Gen--AI.git>